The router — how a route is found

`docs/web/route/router.js`, with `route.js` and `polar.js` under it. It reads the files described in `GRID.md` and `COURSE.md` and answers one question:

> With this boat, this wind and this stream, starting at this time, what is the earliest she could finish, and by what track?

It is not a tactician. It knows nothing of other boats, of sail changes, of a crew that has been up all night, or of the difference between a wind forecast and a wind. What it gives is a *bound* and a *shape*: no boat with that polar could have done better in that weather, and this is where the fastest water was.

Built from the bottom, in four layers

An isochrone router draws a plausible track from almost any arithmetic, which makes it one of the hardest kinds of program to tell is wrong. So each layer was finished and checked while it was still small enough to check by hand.

	layer	what it does	its check
1	`ROUTE.vec`, `advance`	the boat's vector plus the stream's; the position after it	`route-vector.js`
2	`POLAR.speed`, `sail`	the boat's speed at an angle and a wind speed	`polar.js`
3	`ROUTE.step`	one time step, from the grids and the polar	`route-step.js`
4	`ROUTER.run`	every heading from everywhere she could be, until one finishes	`router.js`

One step

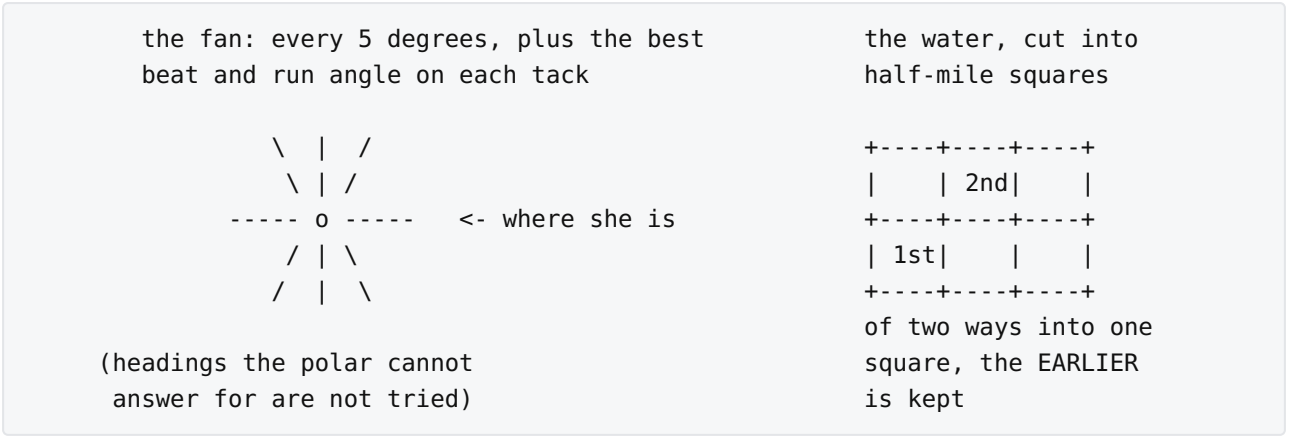
```
            wind over the ground, 12 kt from the west
      ------------------------------------------------>
      --------->  the stream, 2 kt, going the same way

      ----------------------------------------->
            wind over the WATER, 10 kt:  the boat's wind
```

The boat sails in the wind over the water. A polar is measured through the water, so it is entered with the ground wind less the stream. With the stream running *against* the wind the two add, which is why wind against tide is windier as well as rougher. Every row of a route carries both winds.

The polar then gives her speed through the water at her angle to that wind; the stream is added to it; and she is moved over the ground for the length of the step. The conditions are those at the start of the step, held for the whole of it.

Beating and running come from the polar file's own optimum rows. A table that starts at 52 degrees says nothing about 45, and a boat that could not sail closer than that could not beat. The file's `beat_angle` and `beat_vmg` rows say where the best upwind angle is and what it makes good, so `POLAR.sail` answers out to that angle and no further; the same deep downwind, where the boat gybes rather than runs square. Nothing is invented: every speed lies between two numbers that are in the file.

The search

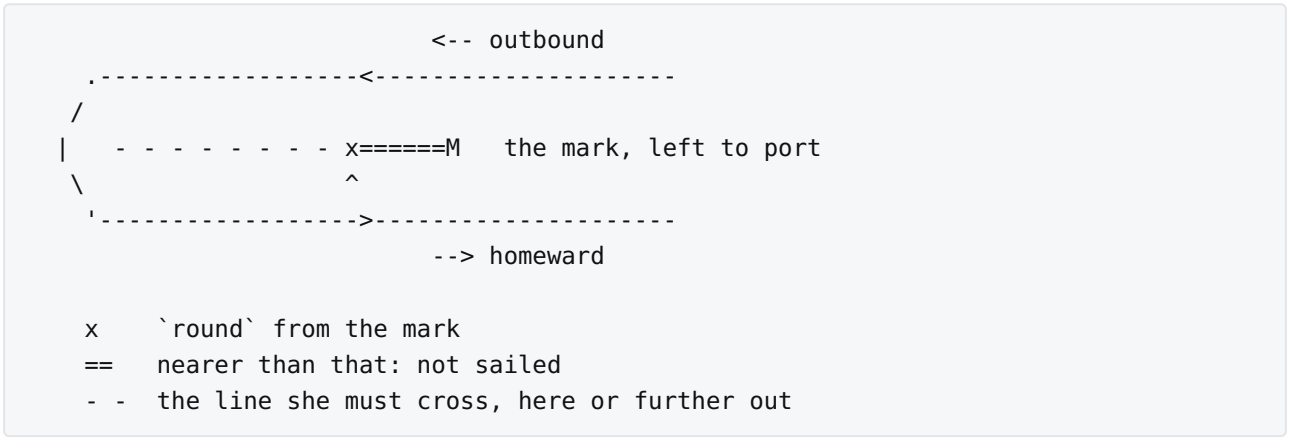


From every place she could be, every heading is tried for one time step. Of all the ways of reaching a half-mile square, only the earliest is kept; of two that arrive in the same step, the one nearer where she is bound. Then the same again from all the places kept, until a step crosses the finish line.

The one assumption in it: that being somewhere sooner is never worse than being there later. That is true of a boat that could always stop and wait, and near enough true of one that cannot. It is what makes the search finish, and it is the ordinary assumption of an isochrone router, stated here rather than left to be found.

When a mark or the finish is within a step, one more heading is tried: the one that lays it exactly. A finish line a tenth of a mile long would otherwise fall between two headings of the fan.

Rounding a mark



The racing rule is a string: drawn taut along the track, it must pass each mark on the required side. The router lays a line out from the mark through the middle of the turn, and she has rounded it when she crosses that line going the right way round — anticlockwise to leave it to port, clockwise for starboard. A track that cuts inside never crosses the line; one that passes on the wrong hand crosses it backwards and is refused.

`round` is a berth (Larry, 13 November 2026): she crosses that line at least `round` miles out and passes no nearer at any time. The fastest route passes exactly that far off. Boats in the real race do go inside what a prudent berth would be, and have for years; a run can be given a smaller number to see what that is worth, and nothing here is advice to do it.

A mark that is the end of the finish line is rounded by finishing, and its side is checked against the way the line is crossed.

The water she may not cross

in the files	what the router does
a `line` in a live layer	never crosses it, and stays the layer’s clearance off it
an `area` in a live layer	never enters it; its edges are lines
`avoid <name> ... off <nm>`	stays at least that far from the point
a layer the run does not name	is not in the water at all
the start line, on the first leg	is a wall behind her

A place on the start line that is already inside a clearance — one end of the 2025 line is a flagstaff ashore — is not a place a boat can start from, and is left out and counted.

What a route says about itself

Every row is one step: where and when it began, the heading and tack, both winds, the stream, the speed through the water and over the ground, and where it ended. The summary adds them up, and says three things a reader should know before trusting the rest:

`noStreamMinutes`	how long she sailed where the atlas has no data, through still water
`clampedMinutes`	how long the wind was above the strongest in the polar
`optimumMinutes`	how long she was on the polar’s beat or run rows rather than its table

The passage log

`docs/web/route/passage.js`. The router answers with one row for every step - 144 of them for the 2025 race - which is right for a chart and too much for a person. The passage log folds those rows into the lines a navigator would have written in the deck log, and beside each line sets down **what had changed** since the line before.

```
 rows     10:00 port 280 | 10:10 port 280 | 10:20 port 281 | ...
            \__________ the same tack, on the same leg _________/
                                  |
 boards   13:00  port, heading 280, for 2h20, 22.8 nm
                                  |
            three or more boards of twenty minutes or less
                                  |
 lines    10:00  short-tacking, 10 boards in 1h40
```

It states facts and never gives a reason (Larry, 13 November 2026). The search keeps the earliest arrival out of every heading tried from every place. It holds no reasons, so a “because” in the log would be invented. The log says that the wind veered 16 degrees between 22:40 and midnight, and that at midnight she tacked to starboard. Whether the one led to the other is for the reader, and the lesson, to say. One of the log’s own checks reads every word the others produced and fails on a reason-word.

An entry is made at a time, as in a deck log. It is laid out down the page, and no line is wider than 80 columns, so it prints and reads on a small tablet without wrapping where the screen chooses:

```
13:00  tacked to port, heading 280   2h20  22.8 nm                    (1)
         wind 242 at 20 kt (21 over the water)                        (2)
         stream fair 0.3 kt   9.5 kt through the water, 9.8 made good  (3)
         since 11:40: wind freshened 4 kt                             (4)

 (1) what she did, and how long and how far the board was
 (2) the wind as the line begins
 (3) the board as a whole: the stream along her track, and her speeds
 (4) what had changed since the line before
```

Every change is the difference of two printed numbers. Each reading is rounded first and subtracted second, so a reader with the log in front of them can check “backed 8” by subtraction: 248 at 10:00, 240 at 11:40.

the log says	when
wind veered / backed N degrees	its direction has moved 5 degrees or more, clockwise or anticlockwise
wind freshened / eased N kt	its speed has changed by 3 knots or more
stream data begins / no stream data from here	the atlas starts, or stops, having data where she is
stream has turned, now setting …	where the stream goes has swung through more than a right angle, and both readings are 0.3 knots or more: near slack water a direction is noise
stream fair / foul / slack on this line, … on the last	the stream along her track is different from the last line's; under a quarter of a knot is slack
tacked / gybed	which end of the wind her bow passed through
bore away / hardened up N degrees	the same tack on a new heading, 5 degrees or more further from the wind or nearer it
short-tacking, N boards	three or more boards together, each of twenty minutes or less

Fair and foul are about her as much as about the water: a tack can make a stream foul that has not changed at all. So that line says what it was and what it is, and not that the stream did it.

The numbers in that table are choices, not physics. They are in one place in the file, they come back with every log so a page can say what they were, and a caller can set them.

What a page gets. `PASSAGE.log(rows)` returns the lines as data: each has its kind, its time, its readings, its changes, and the first and last row it covers, so a page can light the piece of track a line is about. `PASSAGE.text()` prints them as above.

Sailing a run

`docs/web/route/sail.js` takes the name of a run file and does the rest.

```
node docs/web/route/sail.js <folder> <run file>
node docs/web/route/sail.js <folder> <run file> --json <file to save>
```

The router is five files that must be called in the right order, and this is the one place that order is written down:

```
the run file            read first, because it names everything else
     |
     +--> the course, each areas file, the wind grid, the current
     |    grid, the polar: each by its own reader
     |
the run is checked against the course and the water it names,
and then against the grids
     |
ROUTER.run  -->  the rows  -->  PASSAGE.log
     |
the report:   what was sailed
              what to know before trusting it
              the passage log
```

The report also says what the run asked her to keep off, so that it can be read without the run file beside it.

Every refusal names the file. A run names six files or more, and “rows 18 but 17 found” is no use without knowing which of them it was said about.

The checks before sailing are the ones that would otherwise route. A current handed in as the wind, or a wind from 925 hPa, does not fail: it sails, wrongly. Both are refused before anything is sailed.

It adds nothing to the router and takes nothing away. The route is the one `ROUTER.run` returns for the same files, row for row, and a check holds it to that.

the command ends with	when
0	the run was sailed; the report is printed
1	it was refused; the reason is printed, and nothing else
2	it was not asked properly; it says how to ask

`--json` saves the summary, the log’s lines and every row of the route, each row with the position at both its ends, which is what a chart needs to draw the track.

A page reaches the router the same way. `SAIL.sail(read, runName)` is given a function that returns a file’s text by name, so a page hands it what it fetched and the command hands it a folder.

This is a beta (Larry, 13 November 2026). The router is handed to the course to be tried before it is finished, so that what gets built next is what the lessons turn out to need. The report says so on its first line, with the two things most likely to mislead: a tack costs the router nothing, and gates are refused rather than sailed.

The first run: the 2025 Myth of Malham

Sailed on the course’s own files — the GFS wind of 00Z on race day, the Shom stream atlas, the low-water coastline kept a tenth of a mile off, a generic 70 ft polar — in about three seconds.

Start, IRC One	10:00Z, 24 May, at the outer end of the line
The wind	248 at 16 knots at the start: **a dead beat to the Eddystone**
Eddystone, left to port	01:14Z, after 15 hours and 27 tacks, 0.13 miles off
Finish	09:42Z, 25 May: **23.7 hours**, 263 miles over the ground
Off Portland Bill	0.21 miles: inside the race, with the tide
Where the atlas is silent	the first 100 minutes, in the Solent

Checked afterwards by brute force, with none of the router's own bookkeeping: no step crosses the coastline, the nearest it comes is 0.146 miles, the traffic scheme is never approached, and the track swings 360 degrees anticlockwise round the lighthouse.

The real Clipper 68s took about 44 hours. The difference is the point, not a fault: this is the fastest a generic 70-footer could go, tacking for nothing, steered perfectly, in a forecast that came exactly true.

Staying a mile off the three headlands cost nothing the router can measure. The same race with `avoid` lines at St Alban's Head, Portland Bill and Start Point finishes within the same minute, a difference smaller than the search itself can resolve. On that day, in that wind, the inside passage at the Bill was not worth having.

The same race, as the passage log has it

Every line below is the log's own output from that run, nothing added.

```
-- 24 May 2025, times UTC --
10:00  START  Myth of Malham 2025, IRC One
10:00  short-tacking, 10 boards   1h40  15.4 nm
         wind 248 at 16 kt (16 over the water)
         no stream data   9.2 kt through the water, 9.2 made good
11:40  starboard, heading 205   1h20  13.6 nm
         wind 240 at 16 kt (17 over the water)
         stream fair 0.9 kt   9.4 kt through the water, 10.2 made good
         since 10:00: wind backed 8 degrees; stream data begins
13:00  tacked to port, heading 280   2h20  22.8 nm
         wind 242 at 20 kt (21 over the water)
         stream fair 0.3 kt   9.5 kt through the water, 9.8 made good
         since 11:40: wind freshened 4 kt
15:20  short-tacking, 3 boards   0h40  5.9 nm
         wind 239 at 19 kt (18 over the water)
         stream foul 0.7 kt   9.7 kt through the water, 8.9 made good
         since 13:00: stream has turned, now setting 028 at 0.7 kt;
           stream foul on this line, fair on the last
16:00  port, heading 271   3h40  32.1 nm
         wind 237 at 20 kt (19 over the water)
         stream foul 0.9 kt   9.7 kt through the water, 8.8 made good
19:40  tacked to starboard, heading 194   1h40  16.2 nm
         wind 231 at 19 kt (19 over the water)
         stream slack   9.5 kt through the water, 9.7 made good
         since 16:00: wind backed 6 degrees;
           stream slack on this line, foul on the last
21:20  tacked to port, heading 276   0h50  8.8 nm
         wind 235 at 22 kt (23 over the water)
         stream fair 0.9 kt   9.7 kt through the water, 10.5 made good
         since 19:40: wind freshened 3 kt;
           stream has turned, now setting 236 at 0.9 kt;
           stream fair on this line, slack on the last
22:10  short-tacking, 3 boards   0h30  5.6 nm
         wind 238 at 22 kt (24 over the water)
         stream fair 1.6 kt   9.6 kt through the water, 11.1 made good
22:40  port, heading 290   1h20  14.6 nm
         wind 241 at 23 kt (25 over the water)
         stream fair 1.2 kt   9.7 kt through the water, 10.9 made good
```

And through the night to the finish:

```
-- 25 May 2025, times UTC --
00:00  tacked to starboard, heading 220   0h30  4.8 nm
         wind 257 at 22 kt (23 over the water)
         stream slack   9.6 kt through the water, 9.5 made good
         since 22:40: wind veered 16 degrees;
           stream slack on this line, fair on the last
00:30  short-tacking, 3 boards   0h30  4.9 nm
         wind 259 at 23 kt (23 over the water)
         stream slack   9.8 kt through the water, 9.8 made good
01:00  starboard, heading 208   0h15  2.5 nm
         wind 262 at 22 kt (22 over the water)
         stream slack   10.7 kt through the water, 10.4 made good
01:15  ROUNDED Eddystone Lt House  15h15 from the start
01:15  bore away 92 degrees, still starboard, heading 110   2h15  32.3 nm
         wind 261 at 23 kt (22 over the water)
         stream fair 0.8 kt   13.5 kt through the water, 14.3 made good
         since 01:00: stream fair on this line, slack on the last
03:30  gybed to port, heading 057   6h12  83.6 nm
         wind 262 at 22 kt (21 over the water)
         stream fair 1.2 kt   12.3 kt through the water, 13.5 made good
09:42  FINISH  23h42 from the start
```

The first line is ten boards in an hour and forty minutes. No crew tacks that often; the router does because a tack costs it nothing, which is the first item in the list below.

What it does not do yet, and what it assumes

1. **A tack costs nothing.** She short-tacks the Solent every ten minutes because nothing says she should not.
2. **No gates.** `avoid` is sailed; `gate ... within` is refused with the reason rather than ignored.
3. **A mark that may be left on either hand** is refused likewise.
4. **The polar is the boat.** A generic polar gives a generic boat's passage.
5. **The forecast is taken as the truth,** and the stream atlas as the stream.
6. **It is not for navigation.** It is for understanding a race after it was sailed, and for asking what a different choice would have cost.

Running the checks

```
node docs/web/tests/route-vector.js     the vector step
node docs/web/tests/polar.js            the polar
node docs/web/tests/route-step.js       one step, against hand-worked answers
node docs/web/tests/router.js           whole routes small enough to sail in your
head
node docs/web/tests/passage.js          the passage log, on rows written by hand
node docs/web/tests/sail.js             a run sailed from its name, and the refusals
```

Each course in `router.js` is laid on the equator, in a wind and a stream that do not change, so that the fastest passage is one triangle and can be worked out: twenty miles on a beam reach at ten knots is two hours, and a dead beat is the distance over the best speed made good. Eight faults were put into

the router one at a time — marks rounded the wrong way, no berth, clearances ignored, an area left open, the finish crossed either way, finishing without rounding, `avoid` dropped, unnamed layers left live — and each is caught.

The passage log is checked on rows written by hand, in round numbers: six ten-minute rows at nine knots are an hour and nine miles, and 242 less 233 is nine degrees backed. Twenty-four faults were put into it one at a time — veered and backed swapped, fair and foul the wrong way round, a shift taken before the numbers are rounded, the board that rounds the mark folded away with the short tacks, a reason written into a line — and each is caught.

The run command is checked against a folder that exists only in memory, so each refusal can be provoked by taking one file out or spoiling one line. Nineteen faults were put into it one at a time — the grids not checked, a refusal that does not name its file, the areas not read, a refusal that ends with 0 — and each is caught.